



## Cloud Native Application Delivery

# Application Delivery Fundamentals

 Copy page

This article discusses the implementation of CI/CD and GitOps to improve application delivery, reduce errors, and enhance operational efficiency in software development.

Imagine working for a leading software company known for its popular mobile banking app. Your team is developing an innovative feature that enables users to transfer money between accounts with just a few taps on their smartphones. This enhancement is key to maintaining competitiveness and ensuring a superior user experience.

## Challenges with Traditional Release Processes

Historically, releasing new features relied on manually building and testing the code. This outdated process was not only time-intensive but also highly prone to errors. Despite thorough testing, bugs could still appear during deployment, leading to delays and frustrating user experiences. The traditional approach had long lead times and posed significant risks to stability.

## Implementing CI/CD for Improved Efficiency

To overcome these challenges, the team adopted a CI/CD (Continuous Integration/Continuous Deployment) strategy to automate building, testing, and deploying the new feature.

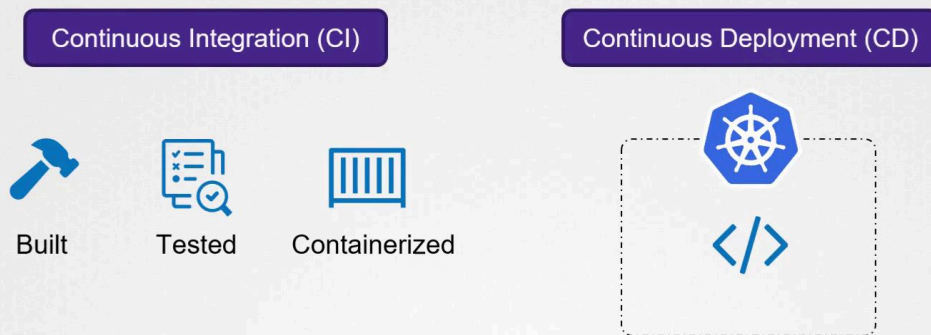
### Continuous Integration (CI):

With CI, any code changes are automatically built, tested, and containerized. This

&gt;

production Kubernetes cluster via a push-based model. This minimizes manual errors and accelerates the release of new features.

## Scenario in the Present

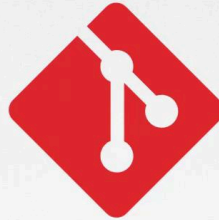


Implementing a CI/CD pipeline helps teams respond quickly to market changes while maintaining high software quality and reliability.

## Leveraging GitOps for Infrastructure Management

Adopting GitOps practices transforms both infrastructure and deployment processes into version-controlled code. This approach enables the team to track every change, significantly reducing risks associated with manual deployments and configuration drift.

&gt;



GitOps

GitOps also simplifies rollbacks. Should any issues arise, reverting changes becomes straightforward, minimizing downtime and ensuring system stability.

## The Pitfalls of Manual Changes

Manual modifications using the command-line interface (CLI) can lead to several issues:

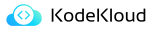
Configuration drift

Increased potential for human error

Overall system instability or failure

Relying on manual changes can complicate disaster recovery. In circumstances such as natural events, technical failures, or human errors, manual configurations hinder the quick restoration of stable systems.

When recovering infrastructure and applications, identifying and replicating manual changes is often time-consuming and error-prone, further delaying system restoration.



&gt;

Provides a single source of truth via Git for both infrastructure and application resources

Enables accurate tracking of every change applied to the system

Allows for rapid rollback of modifications when issues are detected

Streamlines the restoration process by reverting to a known, stable configuration

This approach not only minimizes risks but also enhances overall operational efficiency, ensuring quicker recovery and smoother system operations in the face of unexpected disruptions.

That's it for this article. See you in the next one.



### Watch Video

[Learn more >](#)

Was this page helpful?

Yes

No

### Cost Management

[< Previous](#)

### What is GitOps

[Next >](#)



>

---